

Математические основы алгоритмов, осень 2020 г.
 Лекция 4: Поиск в ориентированном графе: нахождение
 компонентов сильной связности. Поиск в графе с весами:
 алгоритм Беллмана–Форда, алгоритм Дейкстры.
 Нахождение минимального остовного дерева графа:
 алгоритм Прима, алгоритм Крускала. Лес
 непересекающихся множеств, нахождение компонентов
 связности в неориентированном графе*

Александр Охотин

18 декабря 2020 г.

Содержание

1	Поиск в ориентированном графе (окончание)	2
1.1	Нахождение компонентов сильной связности	2
2	Поиск в графе с весами	3
2.1	Алгоритм Беллмана–Форда	3
2.2	Алгоритм Дейкстры. Очередь с приоритетами	5
3	Поиск в графе с весами (окончание)	7
3.1	Реализация очереди с приоритетами с помощью кучи	7
4	Нахождение минимального остовного дерева графа	8
4.1	Какое ребро можно добавить?	8
4.2	Алгоритм Прима (Ярника–Прима–Дейкстры)	8
4.3	Алгоритм Крускала	10
4.4	Правильность алгоритма Крускала	10
5	Структура данных для непересекающихся множеств	11
5.1	Абстрактная структура данных для непересекающихся множеств	11
5.2	Лес непересекающихся множеств	11
5.3	Алгоритм, находящий компоненты связности	13

*Краткое содержание лекций, прочитанных студентам 1-го курса СПбГУ, обучающимся по программам «Математика» и «МАиАД», в осеннем семестре 2020–2021 учебного года. Страница курса: http://users.math-cs.spbu.ru/~okhotin/teaching/algorithms_2020/.

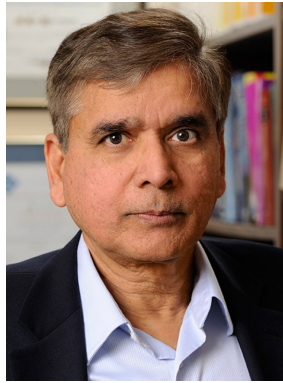


Рис. 1: Рао Косараджу (род. 1943), Миха Шарир (род. 1950).

1 Поиск в ориентированном графе (окончание)

1.1 Нахождение компонентов сильной связности

Ставится задача найти в данном ориентированном графе $G = (V, E)$ его компоненты сильной связности.

Поиск в глубину как таковой компоненты сильной связности не выдаёт. Например, для первой выбранной вершины он просто выдаст все вершины, из неё достижимые — то есть, все компоненты связности, лежащие в графе компонентов связности ниже исходного компонента.

Задача решается алгоритмом Косараджу–Шарира; этот алгоритм был опубликован Шариром [1981], у Косараджу же был неопубликованный черновик.

Алгоритм работает так. Сперва запускается поиск в глубину для G . Затем запускается поиск в глубину для *обращённого графа* G^R , в котором направления всех дуг изменены на обратные¹. При поиске в глубину для G^R , во внешнем цикле вершины рассматриваются *в порядке времени их завершения* при первом поиске в глубину, *от конца к началу*. После этого оказывается, что каждый запуск процедуры DFS во внешнем цикле будет находить очередной сильно связный компонент исходного графа.

Правильность алгоритма основана на следующем свойстве.

Лемма 1. Пусть в графе G есть сильно связанные компоненты C и D , и есть дуга $(u, v) \in E$ из C в D , как показано на рис. 2(левом). Тогда при поиске в глубину в графе G самое позднее время завершения вершины в C превосходит таковое в D .

$$\max_{x \in C} \text{конец}(x) > \max_{y \in D} \text{конец}(y)$$

Доказательство. Рассматриваются два случая, в зависимости от того, который из компонентов, C или D , был обнаружен при поиске первым.

- Пусть самое раннее время обнаружения в C меньше, чем в D . Пусть первой была обнаружена вершина $x \in C$. Тогда в этот момент есть путь из x в любую вершину из $C \cup D$, состоящий из непомеченных вершин («белый путь»). Поэтому при рассмотрении x все эти вершины обрабатываются в рекурсивных вызовах, получая меньшее время окончания, чем у x .

¹Стоит упомянуть, что у графов G и G^R одни и те же компоненты сильной связности.

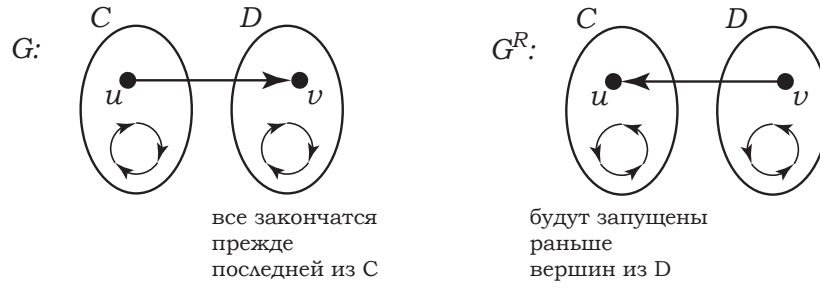


Рис. 2: Сильно связные компоненты, соединённые дугой: (слева) в графе G ; (справа) в графе G^R .

- Если же самое раннее время обнаружения в C больше, чем в D , пусть $y \in D$ — вершина D , обнаруженная раньше других. В этот момент все остальные вершины из D ещё не помечены, и они будут обнаружены в рекурсивных вызовах; вершины же из C на этом этапе обнаружены не будут, потому что до них нет пути. Поэтому время окончания всех вершин из C больше, чем время окончания любой вершины из D .

□

Стало быть, все вершины из D будут закончены раньше, чем вершина из C , законченная последней. Поэтому когда поиск в глубину будет запущен в графе G^R , и первой будет рассмотрена позже всех законченная вершина из C , из неё не удастся прийти в D , поскольку в графе G^R есть только дуга (v, u) , а будь в нём какие-то дуги из C в D , это был бы один компонент связности. Позднее, когда будут рассмотрены вершины из D , переходить из них в C будет уже поздно, поскольку все вершины из C к тому времени будут уже рассмотрены.

Теорема 1. *Вершины каждого дерева, найденного алгоритмом Косадаржу–Шарира при втором поиске в глубину — это и есть сильно связные компоненты исходного графа.*

Доказательство. Индукция по количеству найденных компонентов связности. Надо доказать, что если первые k выданных компонентов связности действительно будут таковыми, то и следующий тоже будет компонентом связности. На этом этапе поиск в глубину в графе G^R находит вершину y , позже всех закончившуюся среди ещё не рассмотренных вершин. Эта вершина, в частности, закончилась позже всех в своём сильно связном компоненте D . Утверждается, что при поиске в G^R из y не удастся пройти ни в какой другой ССК. Действительно, если можно пройти в C по дуге (v, u) , то в графе G была дуга (u, v) , и потому, согласно лемме 1, самое позднее время завершения в C — позднее чем в u . Поэтому все вершины из C к этому моменту уже рассмотрены, туда больше не перейти. □

Время работы алгоритма — $O(|V| + |E|)$.

2 Поиск в графе с весами

2.1 Алгоритм Беллмана–Форда

Пусть в ориентированном графе $G = (V, E)$. для каждой дуги $(u, v) \in E$ задан её *вес* — число $w_{u,v} \in \mathbb{R}$. Если дуги нет, можно положить $w_{u,v} = \infty$.

Нужно найти *пути наименьшего веса* из данной вершины $s \in V$ во все вершины графа.

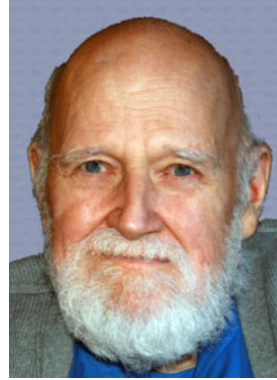


Рис. 3: Ричард Беллман (1920–1984), Лестер Форд мл. (1927–2017).

Стоит заметить, что в графе допускаются дуги отрицательного веса, и такие дуги могут сократить уже проделанный путь. Если же в графе есть *цикл отрицательного веса*, то тогда, проходя его много раз, можно сделать вес пути сколь угодно малым — и потому путь наименьшего веса окажется неопределённым.

Поэтому алгоритм ищет пути наименьшего веса из s во все вершины, коль скоро из s нельзя придти ни в какой *цикл отрицательного веса*. Если же это условие неверно, то алгоритм должен об этом сообщить.

Алгоритм Беллмана–Форда решает эту задачу, вычисляя для каждой вершины $v \in V$ следующие значения:

- d_v : наименьший вес пути из s в v ;
- π_v : предыдущая вершина на пути наименьшего веса из s в v .

Значения π_v образуют остовное дерево.

Изначально полагается $d_v = \infty$, $\pi_v = \text{NULL}$ для всех вершин, и $d_s = 0$. Это первое приближение: найдена только начальная вершина, пути в другие вершины неизвестны.

Алгоритм постепенно находит пути меньшего веса в другие вершины, запоминая веса лучших из найденных путей в этих переменных. Значения уменьшаются с помощью элементарной операции *улучшения пути*, используя некоторую дугу $(u, v) \in E$.

- 1: **if** $d_u + w_{u,v} < d_v$ **then**
- 2: $d_v = d_u + w_{u,v}$
- 3: $\pi_v = u$

Алгоритм 1 применяет эту операцию, пока можно что-то улучшить. Как будет показано ниже, для этого достаточно просмотреть все дуги $|V| - 1$ раз.

Лемма 2. После i -й итерации внешнего цикла алгоритм Беллмана–Форда находит все пути наименьшего веса длины не более чем i .

Доказательство. Индукция по i .

Базис: $i = 0$. Вес пути длины 0 равен 0.

Индукционный переход. Пусть верно для i -го прохода. Всякий путь наименьшего веса длины $i + 1$ в вершину v на предыдущем шаге проходит через некоторую вершину u , и потому содержит путь наименьшего веса длины i в u . Тогда на $(i + 1)$ -м проходе искомый путь в вершину v будет найден при рассмотрении вершины u .

Алгоритм 1 Алгоритм Беллмана–Форда

```
1:  $d_s = 0$ 
2:  $d_v = \infty$  для всех  $v \neq s$ 
3: for  $i = 1$  to  $|V| - 1$  do
4:   for all  $(u, v) \in E$  do
5:     if  $d_u + w_{u,v} < d_v$  then /* если путь из  $s$  в  $v$  можно улучшить этой дугой */
6:        $d_v = d_u + w_{u,v}$ 
7:        $\pi_v = u$ 
8: for all  $(u, v) \in E$  do
9:   if  $d_u + w_{u,v} < d_v$  then
10:    return есть достижимый цикл отрицательного веса
11: return достижимых циклов отрицательного веса нет
```

□

Теорема 2. Алгоритм Беллмана–Форда за $|V| \cdot |E|$ шагов или правильно вычисляет пути наименьшего веса из вершины s во все вершины, или сообщает о наличии достижимого цикла отрицательного веса.

Доказательство. После $|V| - 1$ итераций, согласно лемме 2, найдены все пути наименьшего веса, состоящие не более чем из $|V| - 1$ дуг. Время работы — поскольку на каждой итерации рассматриваются все дуги.

Если после $|V| - 1$ итераций первого цикла оказывается, что какие-то пути можно ещё улучшить, это значит, что есть путь длины $|V|$ в какую-то вершину v , вес которого меньше, чем у всех путей длины не более чем $|V| - 1$, идущих в v . Какая-то вершина x на этом пути повторяется дважды. При удалении фрагмента пути между двумя вхождениями x получается путь длины не более чем $|V| - 1$, который, по предположению, имеет больший вес, чем исходный путь длины $|V|$. Стало быть, вес удалённого цикла отрицательный, о чём и сообщит алгоритм.

Пусть после $|V| - 1$ итераций никакие пути улучшить уже нельзя. Утверждается, что всякий цикл $v_0, v_1, \dots, v_{k-1}, v_0$, достижимый из s , будет иметь неотрицательный вес. Действительно, для каждой дуги (v_i, v_{i+1}) этого цикла условие улучшения пути не выполняется.

$$d_{v_i} + w_{v_i, v_{i+1}} \geq d_{v_{i+1}}$$

(арифметика с номерами вершин — по модулю k) Эти неравенства складываются по всем i .

$$\sum_{i=0}^{k-1} d_{v_i} + \sum_{i=0}^{k-1} w_{v_i, v_{i+1}} \geq \sum_{i=0}^{k-1} d_{v_i}$$

Отсюда следует, что вес цикла $\sum_{i=0}^{k-1} w_{v_i, v_{i+1}}$ неотрицателен. □

2.2 Алгоритм Дейкстры. Очередь с приоритетами

Алгоритм Дейкстры [1959] (Dijkstra) — другой алгоритм для решения той же задачи, работающий быстрее, но требующий, чтобы веса всех дуг были неотрицательными.

Основное отличие алгоритма Дейкстры от алгоритма Беллмана–Форда состоит в том, что алгоритм Дейкстры на каждом шаге находит очередную вершину u , путь наименьшего



Рис. 4: Эдсгер Дейкстра (1930–2002).

веса в которую к этому моменту уже известен. Тогда алгоритм рассматривает все дуги, исходящие из вершины u . Это позволяет ему просматривать каждую дугу графа лишь однажды.

На каждом шаге алгоритм помнит набор вершин, пути наименьшего веса из которых ещё не продолжены. Обозначение: Q . Изначально это все вершины, $Q = V$.

Алгоритм 2 Алгоритм Дейкстры

```

1:  $d_s = 0$ 
2:  $d_v = \infty$  для всех  $v \neq s$ 
3:  $Q = V$ 
4: while  $Q \neq \emptyset$  do
5:     пусть  $u \in Q$  — вершина с минимальным значением  $d_u$ 
6:      $Q = Q \setminus \{u\}$ 
7:     for all  $v: (u, v) \in E$  do
8:         if  $d_u + w_{u,v} < d_v$  then /* если путь из  $s$  в  $v$  можно улучшить этой дугой */
9:              $d_v = d_u + w_{u,v}$ 
10:             $\pi_v = u$ 

```

Теорема 3. *Алгоритм Дейкстры работает правильно.*

Доказательство. Пусть D_v — наименьший вес путей из s в v .

Утверждение о правильности: в начале каждой итерации внешнего цикла, для всякой вершины $v \notin Q$, путь наименьшего веса в неё уже построен.

Индукция по длине вычисления. В начале работы условие выполняется, поскольку вершин не из Q нет. Пусть из Q удаляется вершина u , и при удалении всех предшествовавших вершин условие выполнялось. Надо доказать, что значение d_u в этот момент — это наименьший вес пути из s в u ; иными словами, что $d_u = D_u$.

Какой-то путь наименьшего веса D_u из s в u есть. Поскольку $s \notin Q$ и $u \in Q$, на этом пути есть пара вершин $x \notin Q$, $y \in Q$ (ничто не мешает вершине x совпадать с s , а вершине y — с u). Вершина x была удалена из Q раньше, чем u , и, по предположению индукции, в тот момент для неё уже был построен путь наименьшего веса. Тогда же этот путь был продолжен до пути наименьшего веса до вершины y (именно это продолжение даёт кратчайший путь в y , поскольку, по предположению, это начальный участок пути наименьшего веса в u).

Тогда известно следующее. Во-первых, раз алгоритм выбрал вершину u из Q , то $d_u \leq d_y$. Во-вторых, $d_y = D_y$, так как путь наименьшего веса в y уже найден. В-третьих, $D_y \leq D_u$, поскольку эти две вершины находятся на одном пути. Из цепочки неравенств $d_u \leq d_y = D_y \leq D_u$ следует, что $d_u = D_u$ — стало быть, путь наименьшего веса в вершину u к этому моменту найден. $\square$

Для представления множества Q алгоритм использует особую структуру данных: *очередь с приоритетами* (priority queue). Каждый элемент Q находится там вместе со своим текущим значением d_v .

Операции:

- $\text{insert}(x)$: вставить новый элемент.
- $\text{min}()$: выдать минимальный элемент.
- $\text{extract_min}()$: выдать минимальный элемент и удалить его.
- $\text{decrease}(x, k)$: изменить значение элемента $x \in Q$ на k , коль скоро k меньше текущего значения x_i .

Сложность алгоритма Дейкстры: $|V|$ раз $\text{extract_min}()$ и $|E|$ раз $\text{decrease}()$, поскольку каждая дуга обрабатывается лишь однажды.

3 Поиск в графе с весами (окончание)

Сложность алгоритма Дейкстры зависит от того, как реализована очередь с приоритетами. Наивная реализация: хранить массив x_v , индексированный по $v \in V$. Тогда decrease работает за время $O(1)$, но extract_min требует время $|V|$. Отсюда общее время работы — $|V|^2 + |E| = O(|V|^2)$.

Другая наивная реализация — хранить сортированный массив. Теперь extract_min работает за время $O(1)$, а для decrease нужно будет переставлять элемент в нужное место, что потребует времени $|V|$. Выйдет время $|E| \cdot |V|$ — ещё хуже.

Как эффективно реализовать очередь с приоритетами?

3.1 Реализация очереди с приоритетами с помощью кучи

Используется куча, в которой значение в каждой внутренней вершине *не больше*, чем значение в любом из её потомков (min-heap).

- $\text{insert}(x)$: добавить в конец кучи новый элемент (он становится листом), после чего дать ему всплыть наверх до его законного места.
- $\text{min}()$: просто вернуть x_1 .
- $\text{extract_min}()$: переместить x_n в x_1 , убрав его в конце; затем запустить исправление кучи из корня, то есть, $\text{heapify}(1)$.
- $\text{decrease}(i, k)$: изменить значение x_i на k , после чего дать элементу x_i всплыть наверх, пока возможно.

Дать элементу x_i всплыть — это значит, что если x_i меньше своего родителя, то он обменивается со своим родителем местами, после чего та же операция применяется к вершине на уровень выше, где теперь находится x_i . И так проделывается до тех пор, пока x_i не доедет до положенного ему места.

4 Нахождение минимального остовного дерева графа

Дан неориентированный связный граф, рёбрам которого сопоставлены числа — веса. Нужно найти одно из остовных деревьев с наименьшим весом. Практический смысл: построение самой дешёвой транспортной сети.

Общий принцип алгоритмов: одно за другим строятся рёбра, принадлежащие некоторому минимальному основному дереву; каждое добавленное ребро — правильное. На каждом шаге работы алгоритма уже построено некоторое подмножество одного из минимальных остовных деревьев, и к нему добавляется одно новое ребро — так, что полученное множество рёбер остаётся подмножеством одного из минимальных остовных деревьев.

4.1 Какое ребро можно добавить?

Сечение графа $G = (V, E)$ — это всякое разбиение множества вершин на C и $V \setminus C$. Ребро $(u, v) \in E$ *пересекает сечение*, если один его конец лежит в C , а другой — в $V \setminus C$.

Минимальное остовное дерево обладает следующим свойством: для всякого сечения графа, среди рёбер, пересекающих это сечение, одно из рёбер с наименьшим весом принадлежит минимальному основному дереву.

Лемма 3. Пусть $G = (V, E)$ — граф, $T \subseteq E$ — одно из минимальных остовных деревьев, $F \subseteq T$ — его подмножество. Пусть $(C, V \setminus C)$, где $C \subseteq V$ — некоторое сечение графа, такое, что никакое ребро из F его не пересекает. Тогда ребро с наименьшим весом, пересекающее это сечение, принадлежит некоторому минимальному основному дереву T' , также содержащему F .

Доказательство. Пусть (u, v) — такое ребро. Если оно принадлежит T , утверждение теоремы верно для $T' = T$. Пусть $(u, v) \notin T$. Цель — перестроить дерево в T' , чтобы ребро в него попало.

Если добавить в T ребро (u, v) , получится цикл. Так как (u, v) пересекает сечение, цикл пересекает его в каком-то другом месте по некоторому ребру (x, y) . Ребро (x, y) убирается, (u, v) добавляется, получается дерево T' . Поскольку (u, v) — одно из рёбер с наименьшим весом, оно весит не больше удалённого ребра (x, y) , и, стало быть, T' — другое минимальное остовное дерево. $\square$

Все алгоритмы так или иначе выбирают на каждом шаге ребро с наименьшим весом, и потому относятся к классу *жадных алгоритмов*. Согласно теореме, для данной задачи жадный алгоритм даёт оптимальное решение.

4.2 Алгоритм Прима (Ярника–Прима–Дейкстры)

Открыт Ярником [1930], через много лет переоткрыт Примом [1957] и Дейкстрой [1959]. В литературе обычно называется просто «алгоритмом Прима».

Похож на алгоритм Дейкстры для нахождения путей из одной вершины. Здесь дерево строится, начиная с произвольно выбранной начальной вершины s . Текущее подмножество — всегда дерево. На каждом шаге к дереву добавляется ребро из одной из уже имеющихся в нём вершин в некоторую ещё не задействованную вершину. Добавление такого ребра не приведёт к образованию цикла. Из всех возможных рёбер выбирается ребро с наименьшим весом.

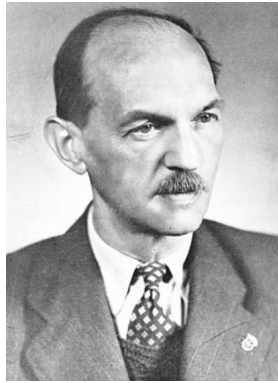


Рис. 5: Войтех Ярник (1897–1970), Роберт Прим (1921–2012).

Структура данных: очередь с приоритетами, в которой хранятся все вершины, ещё не попавшие в дерево. Значение d_v каждой вершины v — это наименьший вес ребра, соединяющий её с деревом. Также для v запоминается вершина дерева π_v , через которую v соединена с деревом ребром наименьшего веса.

Всякий раз, когда в дерево добавляется новая вершина u , для любой вершины v не из дерева может оказаться, что ребро (u, v) легче, чем ранее известное ребро наименьшего веса (π_v, v) , соединяющее её с деревом: это так, если $w_{u,v} < d_v$. В этом случае значения d_v и π_v обновляются, переключая v на соединение с деревом через u .

Алгоритм 3 Алгоритм Прима (Ярника–Прима–Дейкстры)

Пусть $s \in V$ — произвольно выбранная начальная вершина.

```

1:  $d_s = 0$ 
2:  $d_v = \infty$  для всех  $v \neq s$ 
3:  $T = \emptyset$ 
4:  $Q = V$ 
5: while  $Q \neq \emptyset$  do
6:   пусть  $u \in Q$  — вершина с минимальным значением  $d_u$ 
7:    $Q = Q \setminus \{u\}$ 
8:   if  $u \neq s$  then
9:     добавить  $(u, \pi_u)$  к  $T$ 
10:  for all  $v: (u, v) \in E$  do
11:    if  $v \in Q$  и  $d_v > w_{u,v}$  then
12:       $d_v = w_{u,v}$ 
13:       $\pi_v = u$ 

```

Выполняется $|V|$ итераций внешнего цикла. Операции над очередью с приоритетами — $\log |V|$. Внутренний цикл по v : за всё время работы алгоритма каждое ребро рассматривается дважды, с одного конца и с другого. Поэтому тело этого цикла в общей сложности выполняется $2|E|$ раз, и всякий раз выполняется операция над очередью с приоритетами за время $O(\log |V|)$. Общее время работы: $|E| \log |V|$.

Упражнение 1. Найти различия между алгоритмом Прима (нахождения остовного дерева наименьшего веса) и алгоритмом Дейкстры (нахождения всех путей наименьшего веса из одной вершины). Построить пример графа, на котором эти алгоритмы, начав работать из одной и той же вершины s , выдают разные деревья.



Рис. 6: Джозеф Крускал (1928–2010).

4.3 Алгоритм Крускала

Текущее подмножество остовного дерева — это лес, то есть, какое-то количество компонентов связности — деревьев. Изначально, компоненты связности — отдельные вершины. На каждом шаге среди всех рёбер, соединяющих разные компоненты связности построенного леса, выбирается ребро с наименьшим весом, и через него объединяются два компонента связности. Эти операции реализуются на структуре данных для непересекающихся множеств — такой, как лес непересекающихся множеств.

Алгоритм 4 Алгоритм Крускала

```
1:  $T = \emptyset$ 
2: for all  $v \in V$  do
3:    $make\_set(v)$ 
4: отсортировать рёбра из  $E$  в порядке увеличения их веса
5: for  $(u, v) \in E$ , в порядке увеличения их веса do
6:   if  $find\_set(u) \neq find\_set(v)$  then
7:      $T = T \cup \{(u, v)\}$ 
8:      $set\_union(u, v)$ 
```

Цикл в строке 5 можно реализовать, предварительно отсортировав все рёбра по весу.

4.4 Правильность алгоритма Крускала

Лемма 4. *На каждом шаге работы алгоритма Крускала переменная T содержит подмножество одного из остовных деревьев минимального веса.*

Доказательство. Индукция по длине вычисления.

Базис: В начале работы T пусто — это подмножество какого-то остовного дерева минимального веса (да и вообще любого дерева).

Переход: Всякий раз, когда алгоритм собирается добавить в дерево очередное ребро (u, v) , для которого u и v лежат в разных компонентах связности, пусть C — это компонент связности, содержащий u . Тогда среди рёбер, пересекающих сечение $(C, V \setminus C)$, вес ребра

(u, v) будет наименьшим; действительно, все рёбра меньшего веса уже были рассмотрены, и либо уже добавлены в T , либо соединяют две вершины из одного компонента связности. Поэтому, согласно лемме 3, после добавления этого ребра вновь получится подмножества одного из наименьших остовных деревьев. $\square$

Теорема 4. При использовании леса непересекающихся множеств алгоритм работает за время $|E| \log |E|$ — и, стало быть, $|E| \log |V|$.

Доказательство. Сортировка займёт время $|E| \log |E|$. Далее алгоритм выполняет $3|E|$ операций над структурой данных, каждая из которых, при реализации через лес непересекающихся множеств, выполняется за время $O(\log n)$, и в общей сложности получается $(|E| \cdot \log |V|)$ — что уже быстрее сортировки.

Если реализовать все улучшения леса непересекающихся множеств, выйдет оценка времени $3|E| \cdot \alpha(|V|) = o(|E| \log |V|)$ — но сортировка всё равно медленней. $\square$

5 Структура данных для непересекающихся множеств

5.1 Абстрактная структура данных для непересекающихся множеств

Абстрактная структура данных для представления разбиения множества на непересекающиеся подмножества. У каждого множества есть выделенный элемент — «представитель». Операции:

- Создать одноэлементное множество $make_set(x)$,
- Найти представителя множества, содержащего данный элемент, $find_set(x)$.
- Объединение двух множеств $set_union(x, y)$,

Как эффективно реализовать эту абстрактную структуру данных?

5.2 Лес непересекающихся множеств

Disjoint set forest, Галлер и Фишер [1964].

Лес из деревьев с путями, ведущими к корню. Каждое дерево соответствует одному из множеств, каждая вершина — его элементам. Элемент в корне — *представитель*. Очевидная реализация такова.

- $make_set(x)$: создаётся дерево, состоящее из одного корня.
- $find_set(x)$: прослеживается путь из x в корень, выдаётся элемент в корне.
- $set_union(x, y)$: корень одного из деревьев станет указывать на корень другого.

В чистом виде это не очень хорошо работает: применяя set_union в неудачном порядке, можно создать деревья-цепочки, и далее поиск будет занимать линейное время. Поэтому используются следующие два улучшения.

1. В каждой вершине хранится её условная сложность — *ранг*. Процедура $make_set(x)$ устанавливает ранг в ноль. При объединении двух деревьев корень дерева меньшего ранга станет указывать на корень дерева большего ранга. Если оба корня имеют одинаковый ранг, то их объединение получит на единицу больший ранг.



Рис. 7: Вильгельм Аккерманн (1896–1962), Роберт Тарьян (род. 1948).

2. При выполнении каждой операции поиска все встреченные на пути вершины перенаправляются в корень для ускорения последующих операций поиска.

Грубая верхняя оценка времени работы такова.

Лемма 5. Пусть всего элементов — n , и к ним применяется t операций. Тогда, с применением первого улучшения, они выполняются за время $O(t \log n)$.

Доказательство. Чтобы эта оценка была справедлива, достаточно одного первого улучшения. Тогда ранг дерева — это просто его высота, и оказывается, что высота каждого дерева не превосходит двоичного логарифма от числа вершин в нём.

Действительно, в дереве высоты h будет не менее чем 2^h вершин. Это доказывается индукцией по h . В дереве высоты 0 — одна вершина. Если дерево высоты h получено соединением двух деревьев высоты $h - 1$, то в нём будет не менее чем $2 \cdot 2^{h-1} = 2^h$ вершин. Если же дерево высоты h получено присоединением к дереву высоты h дерева меньшей высоты, то уже в одном из кусков есть хотя бы 2^h вершин.

Поэтому каждая операция поиска будет выполняться заведомо не медленнее, чем за $\log_2 n$ шагов. $\square$

Оценить время работы точнее можно через функцию, обратную к функции Аккермана. Функция Аккермана $A(k, n) = A_k(n)$, определяется так: $A_0(n) = n + 1$, $A_k(0) = A_{k-1}(1)$ и $A_k(n) = A_{k-1}(A_k(n - 1))$. Отсюда, в частности, следует, что $A_k(n) = A_{k-1}^{n+1}(1)$, где $f^m(n)$ означает $f(f(\dots f(n) \dots))$. Например, $A_1(0) = A_0(1) = 2$ и $A_1(n) = A_0^{n+1}(1) = n + 2$. Далее, $A_2(n) = A_1^{n+1}(1) = 2n + 3$, $A_3(n) = A_2^{n+1}(1) = 2^{n+3} - 3$, $A_4(n)$ — это башня из экспонент высоты n , а потом идёт совсем уж безумный рост. Функция $f(n) = A_n(n)$ растёт очень быстро — заведомо быстрее любой башни из экспонент.

Функция $\alpha(n)$ — это обратная функция к $f(n)$. Она растёт очень медленно, и для всех значений количества элементов n , которые могут встретиться на практике, $\alpha(n)$ не превосходит 4.

Утверждение 1 (Тарьян [1979]). Пусть всего элементов — n , и над ними выполняется t операций. Тогда, с применением первого и второго улучшений, эти операции выполняются за совокупное время $t\alpha(n)$.

5.3 Алгоритм, находящий компоненты связности

Дан неориентированный граф — возможно, несвязный. Надо определить его компоненты связности и научиться быстро отвечать на вопрос о том, принадлежат ли две данные вершины графа одному и тому же компоненту связности.

Нетрудно написать алгоритм, находящий компоненты связности данного графа в виде структуры данных для непересекающихся множеств (алгоритм 5). После этого можно будет проверять, принадлежат ли вершины u и v одному компоненту связности, просто сравнивая $find_set(u)$ и $find_set(v)$.

Алгоритм 5 Нахождение компонентов связности

```
1: for all  $v \in V$  do
2:    $make\_set(v)$ 
3: for all  $(u, v) \in E$  do
4:   if  $find\_set(u) \neq find\_set(v)$  then
5:      $set\_union(u, v)$ 
```

Всего алгоритм выполняет $3|E|$ операций над структурой данных. Если в реализации используется лес непересекающихся множеств, то время работы алгоритма оценивается как $3|E|\alpha(|V|)$.

Список литературы

- [1959] E. W. Dijkstra, “A note on two problems in connexion with graphs”, *Numerische Mathematik* 1 (1959), 269–271.
- [1964] B. A. Galler, M. J. Fischer, “An improved equivalence algorithm”, *Communications of the ACM*, 7:5 (1964), 301–303.
- [1930] V. Jarník, “O jistém problému minimálním”, *Práce Moravské Přírodovědecké Společnosti*, 6 (1930), 57–63.
- [1957] R. C. Prim, “Shortest connection networks and some generalizations”, *The Bell System Technical Journal*, 36:6 (1957), 1389–1401.
- [1981] M. Sharir, “A strong connectivity algorithm and its applications to data flow analysis”, *Computers and Mathematics with Applications*, 7:1 (1981), 67–72.
- [1979] R. E. Tarjan, “A class of algorithms which require nonlinear time to maintain disjoint sets”, *Journal of Computer and System Sciences*, 18:2 (1979), 110–127.